

Technical Report: HR Automation Platform Backend

Candidate: Akithya Imsari

Position: AI Engineer Intern, ZeloraTech Pvt Ltd.

Date: 22/05/2026

Overview

This project implements a multi-agent HR automation backend using FastAPI, SQLite, and modular architectural principles. The system accepts natural-language HR requests, classifies intent, routes to specialist agents, manages both short-term (STM) and long-term memory (LTM), and maintains an append-only audit log.

Architecture

The application is structured to separate routing, agent logic, and data validation:

- **app/routers** — FastAPI route handlers for the API endpoints (Health, Requests, Audit, Memory).
- **app/agents** — Core LLM orchestration, intent classification, and specialist agent business logic.
- **app/schemas** — Pydantic models for request validation and structured response formatting.
- **app/models** — Database interaction, SQLite table definitions, and query execution.

Agent Model

The system includes four specialist sub-agents, each responsible for a specific HR domain:

- **LeaveAgent** — Handles vacation, sick leave, and PTO requests.
- **SchedulingAgent** — Manages meetings, interviews, and shift bookings.
- **ComplianceAgent** — Answers questions about policies, regulations, and code of conduct.
- **ClarificationAgent** — Handles ambiguous requests by asking targeted follow-up questions.

The central orchestrator coordinates these sub-agents, routing the user to the correct specialist based on intent classification without embedding their specific business logic.

Orchestration

The request pipeline follows a sequential, deterministic flow:

1. Normalize the incoming natural language request.
2. Load STM (chat history) and LTM (permanent facts) context for the user ID.

3. Classify intent (with confidence scoring) via the LLM.
4. Route to the appropriate specialist agent.
5. Execute the specialist agent with the injected historical context.
6. Persist the updated STM to the database.
7. Append the transaction to the Audit Log.

LLM Integration

Google Gemini (via google-generativeai) is used for:

- Intent classification and confidence scoring.
- Grounded response generation for specialist agents.
- Identifying missing parameters (e.g., prompting the user for a specific date if missing).

Note on Configuration: The system was adapted to utilise Gemini for high reliability with structured JSON outputs and cost-free local development. The environment checks dynamically read the GEMINI_API_KEY.

Memory

- **STM (Short-Term Memory)** — Stores recent session interactions (user and assistant messages) so agents can understand conversational follow-up questions.
- **LTM (Long-Term Memory)** — Stores permanent, key-value facts about a user (e.g., department, role) managed via dedicated REST endpoints.
- Both memory tiers are injected into the Gemini context window before the agent generates a final response.

Audit Log

Audit records store:

- user_id
- request
- intent
- confidence
- agent
- response
- timestamp

The database enforces append-only behavior to protect the integrity of the historical HR logs.

Interactive Console Demo

In addition to the REST API, the system includes an interactive console interface (cli.py) that allows direct interaction with the HR agent from the terminal — without requiring Postman, a browser, or the FastAPI server to be running.

How to Run

```
python cli.py
```

Features

- Prompts for an Employee ID on startup and loads any existing LTM for that user.
- Accepts natural language HR requests and displays the intent label, confidence score, agent used, and full response — all in real time.
- Shows a memory context injected notice when historical data is retrieved and used.
- Supports inline commands: memory, audit, set, clear stm, switch, help, quit.

Console Commands

Command	Description
memory	Show current STM and LTM for the active user
audit	Show the last 10 audit log entries
set <key> <val>	Save a long-term memory fact (e.g. set department Engineering)
clear stm	Wipe short-term conversation memory
switch	Switch to a different employee ID
help	Show all available commands
quit / exit	Exit the console demo

Note: cli.py talks directly to the project modules. The FastAPI server (run.py) does not need to be running when using the console demo.

Bug Identification & Fixes

During development and testing, the following issues from the initial architecture were resolved:

- **Health Check Dependency** — The /health endpoint falsely reported a 'degraded' state by checking for legacy Anthropic keys. It was refactored to correctly validate the active Gemini environment variable.

- **Schema Import Pathing** — Fixed a `ModuleNotFoundError` during startup caused by a broken import for `app.schemas` in the health router, allowing the application to boot smoothly and pass all UI health checks.

Trade-offs

- **Model Selection** — The system uses Google Gemini rather than Groq/Anthropic. This required refactoring the LLM initialisation but resulted in highly reliable JSON routing and a massive context window for memory injection.
- **Database Limits** — SQLite remains appropriate for this assessment to allow for zero-configuration local execution, but it is not intended for high-concurrency production deployments (where PostgreSQL would be standard).
- **Mock Data** — The current agent business logic intentionally relies on mock data and polite assumptions to fulfil the project specifications without requiring live integration into external HR platforms.